

3. Object Interaction

Author -- Yucheng Wu

Version -- 26.1.25

Example 3.1

```
// Class for a two-digit number display (partial example from book)
public class NumberDisplay {
    private int limit;
    private int value;

    public NumberDisplay(int rolloverLimit) {
        limit = rolloverLimit;
        value = 0;
    }

    // Additional methods like getValue, setValue, etc.
}
```

3.1. Abstraction

Definition

Abstraction is the ability to ignore details of parts to focus attention on a higher level of a problem.

Features

- Allows handling complexity by dividing problems into sub-problems (divide and conquer).
- Treats solved sub-problems as single building blocks without revisiting internal details.
- Used in software to view components like objects at higher levels, e.g., treating a car engine as a single entity while ignoring its internal parts like spark plugs.

Example

```
// In the clock example, abstract from individual digits to two separate two-digit
// displays (hours and minutes)
NumberDisplay hours = new NumberDisplay(24); // Abstracts hours as a display rolling
// over at 24
NumberDisplay minutes = new NumberDisplay(60); // Abstracts minutes as a display rolling
// over at 60
```

- Here, the NumberDisplay class abstracts the behavior of a limited counter, allowing the ClockDisplay to use it without managing digit details directly.

Remark

- Complements modularization; essential for complex systems like car design or software, where no single person can know all details.
-

3.2. Modularization

Definition

Modularization is the process of dividing a whole into well-defined parts that can be built and examined separately, and that interact in well-defined ways.

Features

- Breaks large problems into smaller, independent modules (e.g., engine, wheels in a car).
- Enables separate teams or individuals to work on modules independently.
- In software, modules are often objects or classes, assembled to form the complete system.

Example

```
// In ClockDisplay, modularize into two NumberDisplay objects
public class ClockDisplay {
    private NumberDisplay hours;
    private NumberDisplay minutes;

    public ClockDisplay() {
        hours = new NumberDisplay(24);
        minutes = new NumberDisplay(60);
    }
}
```

- The clock is divided into hour and minute modules, each handled by a NumberDisplay, allowing independent implementation and interaction via method calls.

Remark

- Used with abstraction to manage complexity; in the clock example, identifies two-digit displays as modules rather than implementing the entire clock in one class.
-

3.3. Object Creation

Definition

Object creation involves instantiating objects from classes using the 'new' keyword, often in constructors or methods.

Features

- Objects are created explicitly and stored via references in variables.
- Allows multiple objects from the same class (e.g., two NumberDisplay instances).
- Constructors initialize object state, such as setting initial values or limits.

Example

```
// Creating NumberDisplay objects in ClockDisplay constructor
public ClockDisplay() {
    hours = new NumberDisplay(24); // Creates hours object with limit 24
    minutes = new NumberDisplay(60); // Creates minutes object with limit 60
    updateDisplay();
}
```

- This creates two separate objects for hours and minutes, each with its own state, enabling the clock to function by cooperating objects.

Remark

- Variables of class types store references (arrows in object diagrams), not the objects themselves; essential for understanding runtime structures.
-

3.4. Object Diagrams

Definition

Object diagrams show the objects and their relationships at one moment in time during execution, presenting the dynamic view of a program.

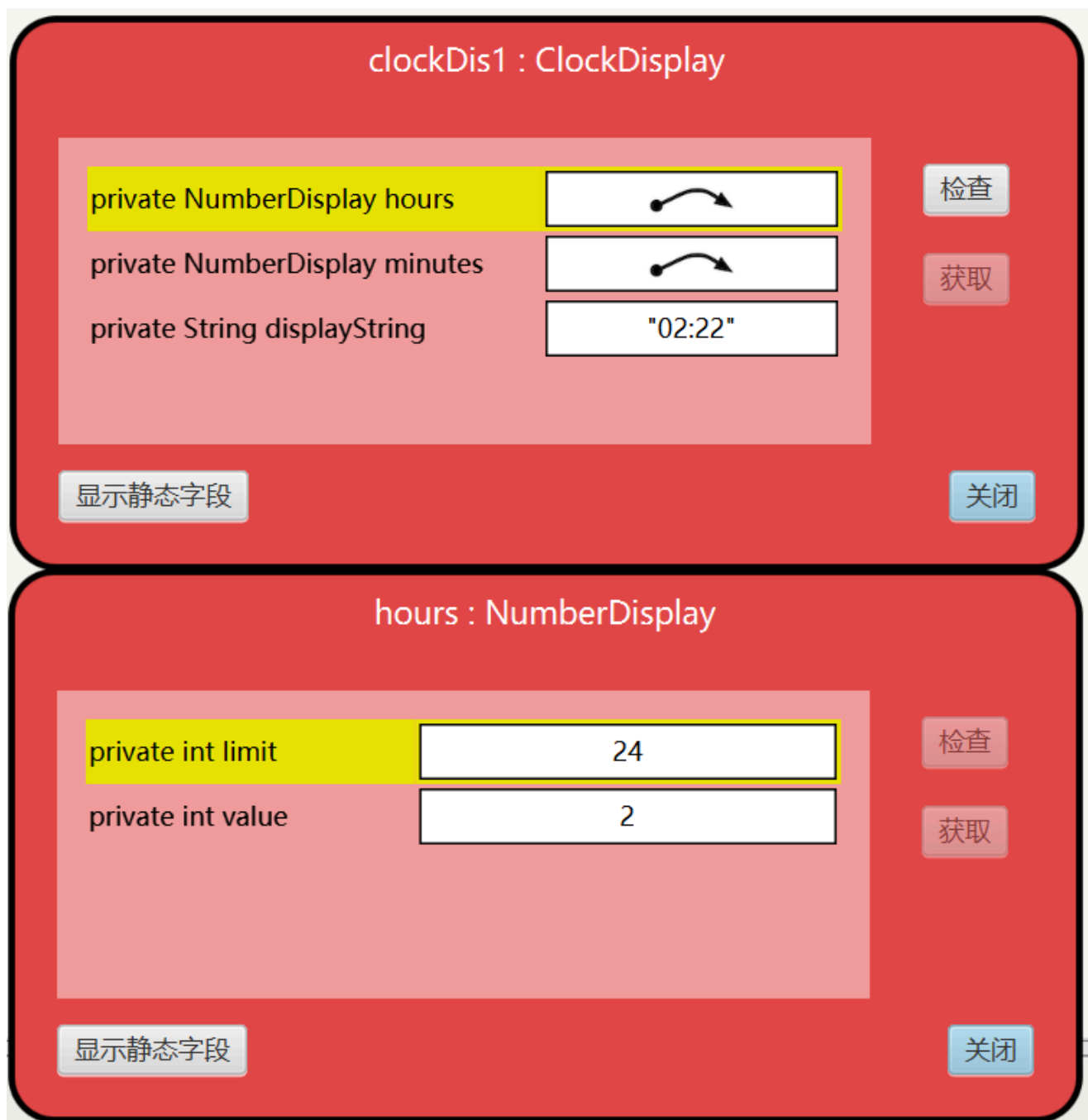
Features

- Depicts runtime instances, including multiple objects from the same class.
- Uses arrows to represent object references.
- Contrasts with class diagrams by showing specific objects rather than general structures.

Example

```
// Runtime structure: One ClockDisplay referencing two NumberDisplay objects
ClockDisplay myDisplay = new ClockDisplay(); // Creates ClockDisplay, which internally
creates two NumberDisplays
```

- Object diagram would show myDisplay (ClockDisplay) with references to hours (NumberDisplay: 11) and minutes (NumberDisplay: 03).



Remark

- Changes during execution as objects are created/destroyed; visualize mentally or on paper to understand program behavior.

3.5. Class Diagrams

Definition

Class diagrams show the classes of an application and the relationships between them, giving information about the source code and presenting the static view.

Features

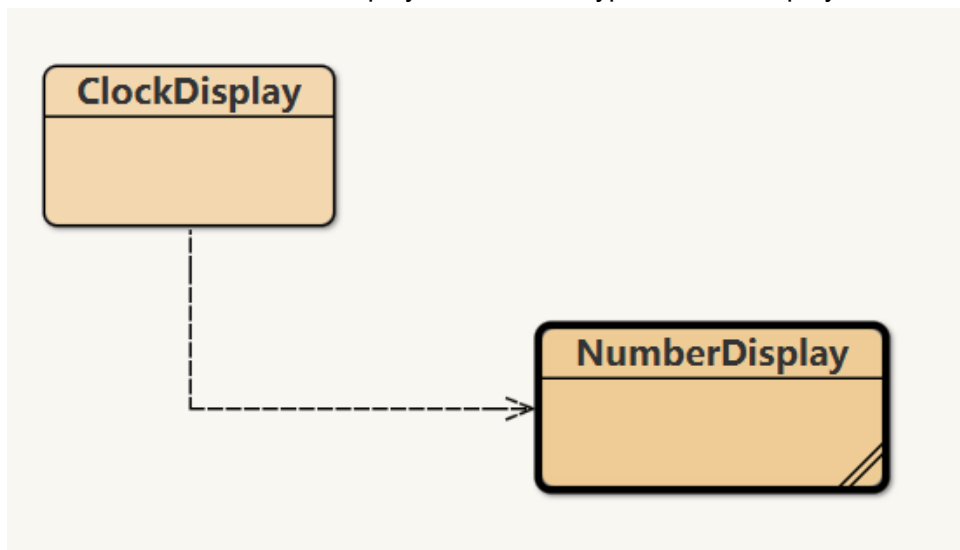
- Illustrates dependencies (e.g., arrows showing one class uses another).
- Does not show multiple instances; focuses on structure at compile time.

- Arrows indicate usage, like ClockDisplay depending on NumberDisplay.

Example

```
// Class diagram: ClockDisplay --> NumberDisplay (dependency)
public class ClockDisplay {
    private NumberDisplay hours; // Reference to NumberDisplay
    private NumberDisplay minutes;
}
```

- Shows two classes: ClockDisplay with fields of type NumberDisplay.



Remark

- Changes when source code is modified; BlueJ displays this view; static and does not reflect runtime instances.

3.6. Logic Operators

Definition

Logic operators (&&, ||, !) operate on boolean values to produce a new boolean, used in conditions like if-statements.

Features

- && (and): true if both operands are true.
- || (or): true if at least one operand is true.
- ! (not): inverts the boolean value.
- Used for compound conditions, e.g., checking value ranges.

Example

```
// In setValue method to validate input
if ((replacementValue >= 0) && (replacementValue < limit)) {
```

```
value = replacementValue;  
}
```

- Ensures value is within 0 to limit-1 before assignment.

Remark

- Careful with operator precedence; can be negated or combined; errors in usage (e.g., replacing && with ||) change behavior significantly.
-

3.7. Primitive Types vs Object Types

Definition

Primitive types (e.g., int, boolean) are basic non-object types with direct value storage; object types are class-defined, storing references.

Features

- Primitives: No methods, predefined (int, boolean, etc.), values stored directly in variables.
- Objects: Have methods, user/system-defined, variables store references (arrows in diagrams).
- Differences in storage, behavior, and usage (e.g., objects created with 'new').

Example

```
// Primitive: int value = 0; // Direct storage  
// Object: NumberDisplay hours = new NumberDisplay(24); // Reference storage
```

- In NumberDisplay, 'value' and 'limit' are primitives; the class itself is an object type.

Remark

- Primitives have no methods; objects enable complex behaviors; see Appendix B for full primitive list.
-

3.8. Modulo Operator

Definition

The modulo operator (%) returns the remainder of division, used for cyclic behaviors like rollovers.

Features

- Expression: $a \% b$ gives remainder of a divided by b .
- Useful for limits, e.g., rolling over counters.
- Works with integers; negative values possible but behavior defined.

Example

```
// In increment method for rollover
public void increment() {
    value = (value + 1) % limit; // Rolls over to 0 when value + 1 == limit
}
```

- For limit 24, $23 + 1 \% 24 = 0$.

Remark

- Key for clock-like behaviors; ensure operands are positive for expected results.
-

3.9. String Concatenation

Definition

String concatenation joins strings using '+', often with other types which are converted to strings.

Features

- '+' operator overloads for strings.
- Automatically converts primitives/objects to strings (via toString() for objects).
- Used for building display strings.

Example

```
// In getDisplayValue to format two-digit string
public String getDisplayValue() {
    if (value < 10) {
        return "0" + value; // Concatenates "0" and int value
    } else {
        return "" + value; // Empty string + value
    }
}
```

- Produces "03" for value 3.

Remark

- Efficient for simple cases; for complex, consider StringBuilder; implicit in System.out.println with multiples.
-

3.10. Operator Overloading

Definition

Operator overloading refers to an operator (like +) having different meanings depending on the types of its operands, such as addition for numbers or string concatenation for strings.

Features

- For numbers: + performs arithmetic addition (e.g., $42 + 12 = 54$).
- For strings: + performs concatenation, joining strings without automatic spaces.
- If one operand is a string and the other is not, the non-string is converted to a string and concatenated.

Example

```
// String concatenation with conversion
String result = "answer: " + 42; // Converts 42 to "42" and concatenates
```

- This produces "answer: 42"; non-strings like integers are automatically converted for concatenation.

Remark

- No spaces are added automatically; include them explicitly if needed. Useful for building strings dynamically, as in `getDisplayValue` method.

3.11. Multiple Constructors (Overloading)

Definition

Overloading allows a class to have multiple constructors or methods with the same name but different parameter lists, providing alternative ways to initialize or perform tasks.

Features

- Constructors differ by parameter types/count.
- Selected based on matching arguments at creation/call.
- Common for flexibility, e.g., default vs. parameterized initialization.

Example

```
// In ClockDisplay class
public ClockDisplay() { // Default: starts at 00:00
    hours = new NumberDisplay(24);
    minutes = new NumberDisplay(60);
    updateDisplay();
}

public ClockDisplay(int hour, int minute) { // Parameterized: set specific time
    hours = new NumberDisplay(24);
    minutes = new NumberDisplay(60);
    setTime(hour, minute);
}
```

- First constructor uses defaults; second allows setting initial time via parameters.

Remark

- No explicit return type; enhances usability. Differences in bodies (e.g., no `updateDisplay` in second if `setTime` handles it).

3.12. Method Calls

Definition

Method calls invoke methods on objects using dot notation, allowing objects to interact and cooperate.

Features

- Uses '.' to call methods (e.g., `object.method()`).
- Can chain calls for complex interactions.
- Involves passing parameters and returning values for tasks like incrementing or getting values.

Example

```
// Increment method calling modulo for rollover
public void increment() {
    value = (value + 1) % limit; // Uses modulo operator for rollover
}
```

- In `ClockDisplay`, `timeTick()` calls `increment()` on minutes, and if rollover, on hours too.

Remark

- Essential for object interaction; debuggers help trace calls; 'this' keyword refers to the current object in methods.

3.12.1. Internal Method Calls

Definition

An internal method call occurs when a method invokes another method within the same class, without dot notation.

Features

- Syntax: `methodName(parameter-list);`
- No object reference needed; implicitly calls on 'this' object.
- Used for code reuse within the class, like updating state.

Example

```
// In ClockDisplay constructor
updateDisplay(); // Internal call to private method
```

- Calls `updateDisplay()` to set `displayString` after creating `NumberDisplays`.

Remark

- Matches signature (name + parameters); supports overloading. Contrasts with external calls.

3.12.2. External Method Calls

Definition

An external method call occurs when an object invokes a method on another object using dot notation.

Features

- Syntax: `object.methodName(parameter-list);`
- Enables object interaction and delegation.
- Public methods typically used; returns values or modifies state.

Example

```
// In ClockDisplay timeTick method
minutes.increment(); // Calls increment on minutes object
```

- ClockDisplay delegates incrementing to the NumberDisplay object for minutes.

Remark

- Uses object reference (e.g., `minutes`), not class name. Essential for cooperation; track with debuggers.
-

3.13. Dot Notation

Definition

Dot notation (`object.method()` or `object.field`) accesses methods or fields of another object.

Features

- Specifies target object before dot.
- Used for external calls/access.
- Chains possible (e.g., `obj.get().method()`).

Example

```
// In ClockDisplay updateDisplay
displayString = hours.getDisplayValue() + ":" + minutes.getDisplayValue();
```

- Accesses `getDisplayValue` on hours and minutes objects.

Remark

- Distinguishes external from internal; errors if object is null.
-

3.14. Class Interface

Definition

A class's interface is the set of public methods it exposes for other objects to use, defining how it interacts without revealing implementation.

Features

- Includes method signatures (name, parameters, return type).
- Enables collaboration; users agree on interface, implement separately.
- Hides internal details via abstraction.

Example

```
// NumberDisplay interface includes:  
public int getValue();  
public void setValue(int replacementValue);  
public void increment();
```

- ClockDisplay uses these without knowing NumberDisplay internals.

Remark

- Promotes modularity; changes to implementation don't affect users if interface unchanged.
-

3.15. The 'this' Keyword

Definition

The 'this' keyword refers to the current object, used to distinguish instance fields from parameters or locals with the same name.

Features

- Accesses fields: `this.field = parameter;`
- Resolves name overloading in constructors/methods.
- Can pass 'this' as parameter or return it.

Example

```
// In MailItem constructor  
public MailItem(String from, String to, String message) {  
    this.from = from; // Assigns parameter to field  
    this.to = to;  
    this.message = message;  
}
```

- Differentiates field 'from' from parameter 'from'.

Remark

- Improves readability by allowing meaningful shared names; avoid if confusing.
-

3.16. Using Debugger

Definition

A debugger is a tool for executing code step-by-step, inspecting variables, and finding bugs by controlling program flow.

Features

- Breakpoints: Stop at specific lines.
- Stepping: Single line (Step) or into methods (Step Into).
- Variable inspection: Locals, instances, statics.

Remark

- Simple in BlueJ; professional ones more advanced. Use for understanding execution paths.
-

3.16.1. Breakpoints

Definition

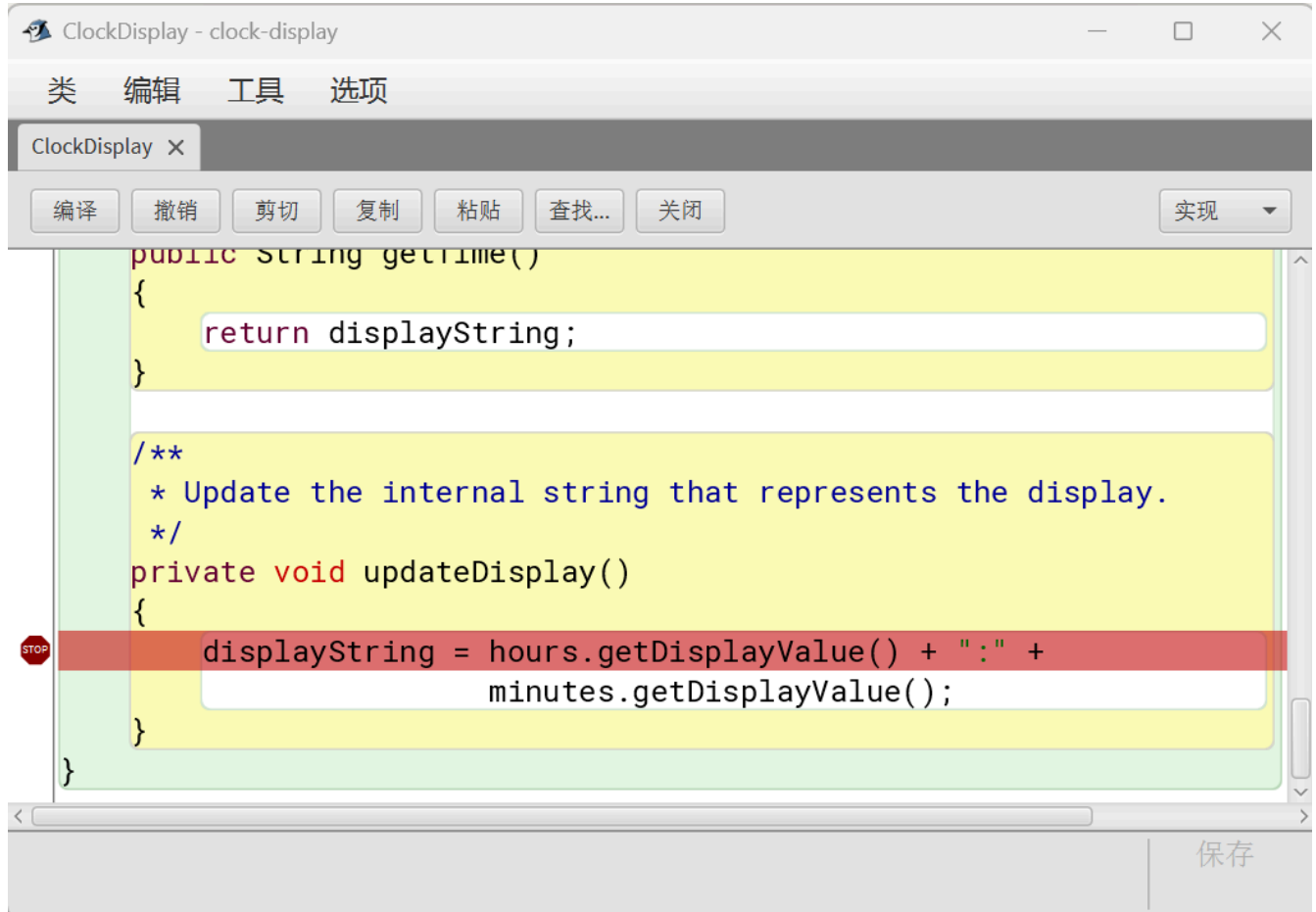
A breakpoint is a marker on a code line that pauses execution when reached, allowing inspection.

Features

- Set/clear in editors (e.g., BlueJ: Tools menu or click).
- Stops before executing the line.
- Resume with Continue, Step, etc.

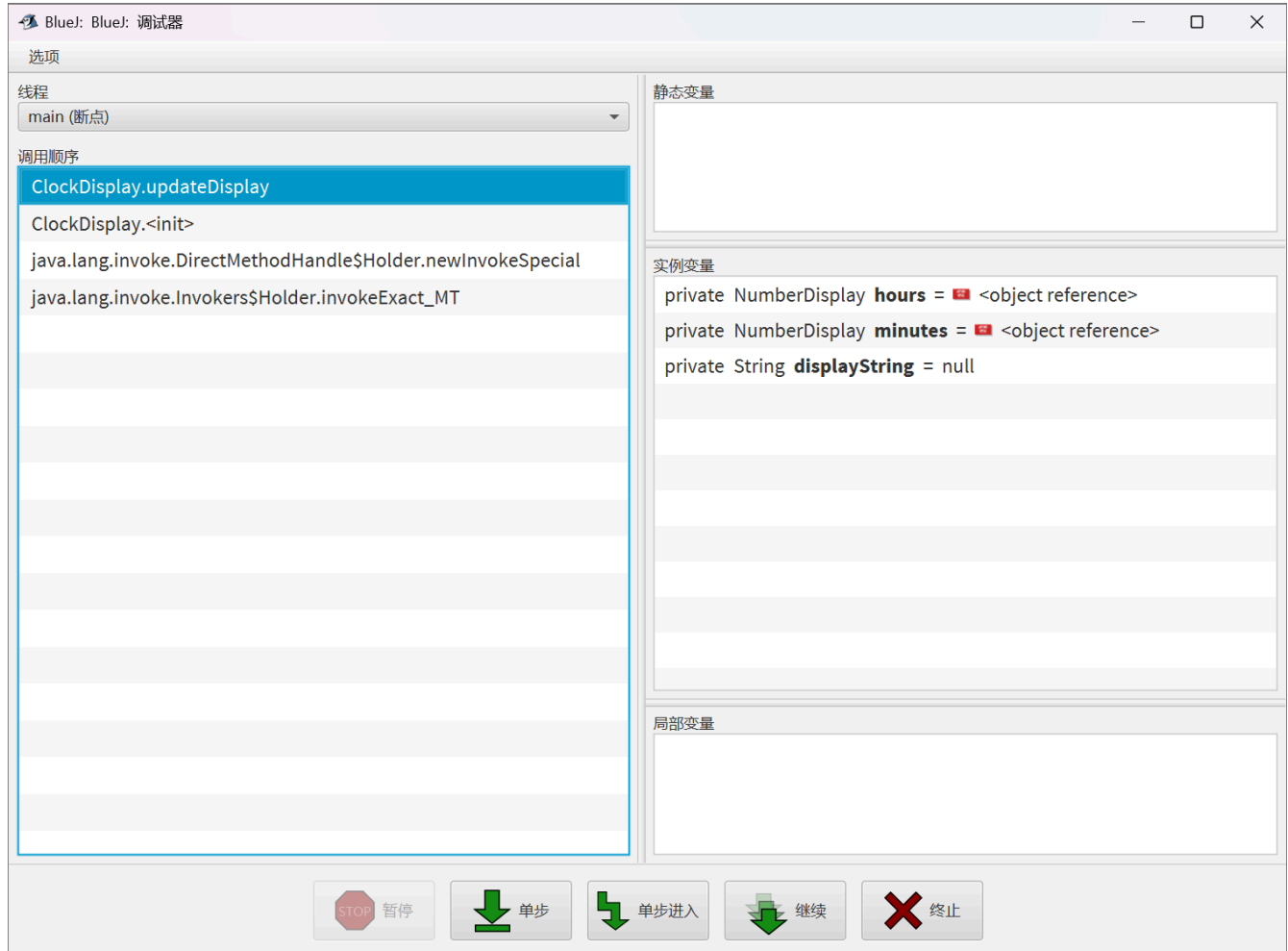
Example

Figure 16.1 A breakpoint in the BlueJ editor.



- Once the **updateDisplay** method was invoke, the debugger window will pop up.

Figure 16.2 The debugger window, execution stopped at a breakpoint.



Remark

- Requires compiled class; useful for conditional paths.

3.16.2. Single Stepping

Definition

Single stepping executes one line of code at a time in a debugger, observing changes.

Features

- 'Step' button (单步): Executes current line, stops at next.
- Tracks flow, especially in loops/conditionals.
- Views variable changes post-execution.

Remark

- Helps verify expectations; repeat calls show state changes.

3.16.3. Stepping Into Methods

Definition

Stepping into enters the called method in the debugger, executing its first line.

Features

- 'Step Into (单步进入)' vs. 'Step' (which treats call as one unit).
- Reveals internal details of abstractions.
- Useful for debugging interactions.

Example

```
// Step Into: item.print();
```

- Enters MailItem's print method, stepping through its lines.

Remark

- Lowers abstraction level; return with 'Step Out' in advanced debuggers.